

Path Planning Assignment

1. Introduction

The purpose of this assignment is to generate a sequence of path points for a car driving through a track marked by colored cones. Yellow cones represent the right boundary and blue cones represent the left boundary. The car pose is provided as an x position, y position, and yaw angle. The planner returns a list of world-coordinate points that represent the route to follow.

The implemented solution deliberately uses simple geometry. This matches the goal of producing an understandable solution without requiring complex optimization, mapping, or machine-learning techniques.

2. Inputs and Output

2.1 Inputs

- CarPose: the current car position (x, y) and yaw angle in radians.
- Cone: each detected cone contains x and y world coordinates and a color flag.
- color == 0: yellow cone, representing the right side of the track.
- color == 1: blue cone, representing the left side of the track.

2.2 Output

The generatePath method returns Path2D, which is a list of (x, y) coordinate pairs. The implementation targets a path length of approximately 8 metres and interpolates segments so that the spacing between consecutive points is no greater than 0.5 metres.

3. Solution Approach

3.1 Vehicle forward direction

The yaw angle is converted into a two-dimensional forward vector. This gives the planner an estimate of the direction in which the car is currently facing. The forward vector is also used to order cone observations and as a fallback when the cone information is insufficient.

3.2 Separating the boundaries

The cones are divided into blue and yellow lists. This makes it possible to distinguish the left boundary from the right boundary before estimating the center of the track.

3.3 Both blue and yellow cones available

When at least one blue cone and one yellow cone are visible, the algorithm pairs cones according to their ordering and calculates the midpoint between each blue-yellow pair. A midpoint is used because it lies halfway between the detected left and right boundaries. These midpoint positions become target points for the path. When two boundary pairs are provided, the resulting midpoints give a simple estimate of the track center line.

3.4 Only one side is visible

If only blue cones or only yellow cones are detected, a true midpoint cannot be calculated because the opposite boundary is missing. The solution therefore estimates the direction of the visible boundary and offsets it toward the expected center of the track.

With two or more visible cones, the first and last ordered cones define an approximate boundary direction. This vector is normalized so that it represents direction only. A perpendicular vector is then calculated from the boundary direction.

The estimated center is obtained by moving each visible boundary cone inward by an assumed half-track width of 1.0 metre. Blue cones are treated as the left boundary and are shifted toward the right. Yellow cones are treated as the right boundary and are shifted toward the left.

3.5 Exactly one cone

A single point cannot define the direction of a boundary. When only one cone is visible, the algorithm therefore uses the vehicle yaw as the estimated boundary direction. The cone is then offset using the same perpendicular-vector method. This is an assumption made because the available sensor information is incomplete.

3.6 No cones

When no cones are detected, the planner cannot estimate either track boundary. In this case, it uses the current vehicle yaw and generates a straight-ahead path. This provides a predictable fallback rather than returning an empty path.

4. Constructing the Final Path

The current car position is used as the initial control point. Estimated center points are then considered for inclusion. The code calculates the projection of each target relative to the car forward vector and ignores targets that are clearly behind the vehicle.

After selecting the useful control points, the algorithm determines a final extension direction. When multiple control points exist, this direction is estimated from the final segment. If no useful cone-based target exists, the vehicle heading is used instead.

The existing control-point path length is calculated and, when necessary, the final point is extended until the desired total length is approximately 8 metres.

4.1 Interpolation

The final control points may be several metres apart. To satisfy the required path resolution, every segment is divided into smaller sections. The number of sections is calculated from the segment length and the 0.5 metre maximum step size. Linear interpolation then generates the intermediate path coordinates.

5. Part 2: Three Cones on One Side

The one-sided logic is written to operate whenever the number of visible cones is at least two. Therefore, the same approach also accepts three cones on one side of the track. The cones are ordered relative to the vehicle forward direction, and the first and last cones are used to estimate the overall direction of that boundary.

Each of the three cones can then be shifted inward by the same perpendicular offset. The resulting three estimated center points provide more observations of the boundary than the original one-cone or two-cone cases.

5.1 Why this approach was chosen

- It reuses the same geometric idea as Part 1, keeping the implementation simple.
- It works with two, three, or more same-side cones without introducing a separate complex algorithm.
- It is deterministic and easy to inspect visually using the supplied tester.
- It is easy to explain and debug because every target point is derived directly from cone geometry.

5.2 Recommended additional test cases

Part 2 requires new tests in `src/scenarios.py`. Suitable cases include:

- Three blue cones forming an approximately straight left boundary.
- Three yellow cones forming an approximately straight right boundary.
- Three same-side cones with changing x and y positions to test a non-axis-aligned boundary.

6. Assumptions

- Cone coordinates are valid world-frame coordinates.
- Blue cones always represent the left boundary and yellow cones always represent the right boundary.
- Yaw is expressed in radians.
- A half-track width of 1.0 metre is a reasonable estimate when the opposite boundary is not visible.
- The car yaw is a useful fallback direction when fewer than two cones define a boundary.
- Cone detections do not contain major outliers or incorrectly classified colors.

7. Limitations

7.1 Fixed width assumption

The 1.0 metre half-track-width assumption may be inaccurate if the real distance from a boundary to the track center changes. This affects cases where only one side is visible.

7.2 Single-cone ambiguity

One cone does not provide enough geometric information to infer the direction of a track boundary. The use of car yaw in this case is only an estimate.

7.3 Cone pairing

When cones exist on both sides, the solution pairs them according to their sorted order. In an irregular layout, the corresponding cones may not always have matching positions, leading to an inaccurate midpoint.

7.4 Sharp direction changes

The final route uses linear segments between control points. If the car heading and estimated center line differ significantly, the resulting path can contain an abrupt change of direction. A spline or curvature-constrained trajectory would produce smoother steering.

7.5 Curved boundaries

Using the first and last cones to estimate a one-sided boundary treats the overall boundary as approximately straight. Three cones on a curved boundary contain additional shape information that this simple method does not fully exploit.

7.6 No history

The planner only uses the current detections. It does not preserve previous cone locations or previous paths, so it cannot use temporal information to stabilize the route.

8. Possible Improvements

- Pair opposite-color cones using nearest-neighbor or geometric matching rather than list order alone.

- Estimate track width dynamically when both boundaries are visible.
- Use all three same-side cones to fit a line or smooth curve instead of relying mainly on the first and last observations.
- Generate a spline or Bezier trajectory to reduce abrupt turns between control points.
- Reject implausible cone observations before generating the path.
- Use information from previous frames to improve path continuity.

10. Conclusion

The solution uses straightforward geometry to estimate a drivable center path from limited cone observations. When both boundaries are visible, midpoint calculation provides a direct center estimate. When only one boundary is visible, the planner estimates a parallel center line using a perpendicular offset. Vehicle yaw provides fallback information when the cone geometry is insufficient. The final path is extended to a useful planning distance and interpolated to maintain the required point spacing.

The approach is intentionally simple and explainable. Its limitations, particularly fixed track width, simple pairing, and linear transitions, are acceptable areas for future improvement rather than hidden assumptions.